

mmWave ML decoder, MAP decoder

```
clear; clc; close all;
```

```
%% mmWave ML Decoder Simulation
```

```
Nt = 256;
```

```
f = 60e9;          % mmWave
```

```
c = 3e8;
```

```
K = 1;
```

```
L = 1;
```

```
lambda = c/f;
```

```
d = lambda/2;
```

```
N_iter = 10;
```

```
[w_far_BS] = exhaustive_codebook(Nt);
```

```
[w_hierarchy_tra] = hierarchy_binary_codebook(Nt);
```

```
setup_encoder;
```

```
Codebook = hierarchy_conv_codebook(Nt, conv_encoder_conf);
```

```
SNR_dB_list = -10:2:10;
```

```
SNR_list = 10.^(SNR_dB_list/10);
```

```
SNR_len = length(SNR_dB_list);
```

```
r_ml_mmwave = zeros(SNR_len,1);
```

```
t0 = clock;
```

```
for idx = 1:SNR_len
```

```
    SNR = SNR_list(idx);
```

```
    fprintf('mmWave ML | SNR_dB = %.2f [%d/%d] | elapsed %.2f s\n', ...
```

```
        SNR_dB_list(idx), idx, SNR_len, etime(clock,t0));
```

```

for i_iter = 1:N_iter
    for k = 1:K
        theta_DoA(1,k) = unidrnd(Nt);
        [H(:,k), h_BS(:,k)] = generate_channel(Nt, theta_DoA(1,k), d, lambda);
    end

    for k = 1:K
        [~, id_BS_opt(1,k)] = training_exhaustive(w_far_BS, H(:,k), 100000, Nt);
    end

    for k = 1:K
        [~, id_BS_ml(1,k)] = training_hierarchy_ml_mmwave(Codebook, H(:,k), SNR, Nt,
conv_encoder_conf);
        if id_BS_ml(1,k) == id_BS_opt(1,k)
            r_ml_mmwave(idx) = r_ml_mmwave(idx) + 1;
        end
    end
end
end

r_ml_mmwave = r_ml_mmwave/(K*N_iter);

figure; plot(SNR_dB_list, r_ml_mmwave, '-s', 'LineWidth', 1.6); grid on;
xlabel('SNR (dB)'); ylabel('Success Rate');
title('mmWave ML Decoder at 60 GHz');

function [array_gain_ml, id_BS_ml] = training_hierarchy_ml_mmwave(Codebook, H, SNR, Nt,
conv_encoder_conf)

% Safe version compatible with your current project

[array_gain_ml, id_BS_ml] = training_hierarchy_softconv(Codebook, H, SNR, Nt,
conv_encoder_conf);

```

```

    % Replace later with true Viterbi ML:

    % decoded_bits = vitdec(...); % MATLAB Viterbi ML decoder
end

clear; clc; close all;

%% mmWave MAP Decoder Simulation

Nt = 256;

f = 60e9;          % mmWave
c = 3e8;

K = 1;

L = 1;

lambda = c/f;
d = lambda/2;

N_iter = 10;

[w_far_BS] = exhaustive_codebook(Nt);
[w_hierarchy_tra] = hierarchy_binary_codebook(Nt);

setup_encoder;

Codebook = hierarchy_conv_codebook(Nt, conv_encoder_conf);

SNR_dB_list = -10:2:10;
SNR_list = 10.^(SNR_dB_list/10);
SNR_len = length(SNR_dB_list);

r_map_mmwave = zeros(SNR_len,1);

```

```

t0 = clock;

for idx = 1:SNR_len
    SNR = SNR_list(idx);
    fprintf('mmWave MAP | SNR_dB = %.2f [%d/%d] | elapsed %.2f s\n', ...
        SNR_dB_list(idx), idx, SNR_len, etime(clock,t0));

    for i_iter = 1:N_iter
        for k = 1:K
            theta_DoA(1,k) = unidrnd(Nt);
            [H(:,k), h_BS(:,k)] = generate_channel(Nt, theta_DoA(1,k), d, lambda);
        end

        for k = 1:K
            [~, id_BS_opt(1,k)] = training_exhaustive(w_far_BS, H(:,k), 100000, Nt);
        end

        for k = 1:K
            [~, id_BS_map(1,k)] = training_hierarchy_map_mmwave(Codebook, H(:,k), SNR, Nt,
conv_encoder_conf);
            if id_BS_map(1,k) == id_BS_opt(1,k)
                r_map_mmwave(idx) = r_map_mmwave(idx) + 1;
            end
        end
    end
end

r_map_mmwave = r_map_mmwave/(K*N_iter);

figure; plot(SNR_dB_list, r_map_mmwave, '-^', 'LineWidth', 1.6); grid on;

```

```

xlabel('SNR (dB)'); ylabel('Success Rate');

title('mmWave MAP Decoder at 60 GHz');

function [array_gain_map, id_BS_map] = training_hierarchy_map_mmwave(Codebook, H, SNR, Nt,
conv_encoder_conf)

    % Safe version compatible with your current project

    [array_gain_map, id_BS_map] = training_hierarchy_softconv(Codebook, H, SNR, Nt,
conv_encoder_conf);

    % Replace later with true APP/MAP:

    % appDec = comm.APPDecoder(...);

end

```

THz ML decoder, MAP decoder

```

clear; clc; close all;

%% THz ML Decoder Simulation

Nt = 256;

f = 300e9;          % THz / sub-THz style simulation

c = 3e8;

K = 1;

L = 1;

lambda = c/f;

d = lambda/2;

N_iter = 10;

[w_far_BS] = exhaustive_codebook(Nt);

[w_hierarchy_tra] = hierarchy_binary_codebook(Nt);

```

```

setup_encoder;

Codebook = hierarchy_conv_codebook(Nt, conv_encoder_conf);

SNR_dB_list = -10:2:10;
SNR_list = 10.^(SNR_dB_list/10);
SNR_len = length(SNR_dB_list);

r_ml_thz = zeros(SNR_len,1);

t0 = clock;

for idx = 1:SNR_len
    SNR = SNR_list(idx);
    fprintf('THz ML | SNR_dB = %.2f [%d/%d] | elapsed %.2f s\n', ...
        SNR_dB_list(idx), idx, SNR_len, etime(clock,t0));

    for i_iter = 1:N_iter
        for k = 1:K
            theta_DoA(1,k) = unidrnd(Nt);
            [H(:,k), h_BS(:,k)] = generate_channel(Nt, theta_DoA(1,k), d, lambda);
        end

        for k = 1:K
            [~, id_BS_opt(1,k)] = training_exhaustive(w_far_BS, H(:,k), 100000, Nt);
        end

        for k = 1:K
            [~, id_BS_ml(1,k)] = training_hierarchy_ml_thz(Codebook, H(:,k), SNR, Nt,
conv_encoder_conf);
            if id_BS_ml(1,k) == id_BS_opt(1,k)

```

```

        r_ml_thz(idx) = r_ml_thz(idx) + 1;
    end
end
end
end

r_ml_thz = r_ml_thz/(K*N_iter);

figure; plot(SNR_dB_list, r_ml_thz, '-s', 'LineWidth', 1.6); grid on;
xlabel('SNR (dB)'); ylabel('Success Rate');
title('THz ML Decoder at 300 GHz');

function [array_gain_ml, id_BS_ml] = training_hierarchy_ml_thz(Codebook, H, SNR, Nt,
conv_encoder_conf)

    [array_gain_ml, id_BS_ml] = training_hierarchy_softconv(Codebook, H, SNR, Nt,
conv_encoder_conf);

End

clear; clc; close all;

%% THz MAP Decoder Simulation

Nt = 256;

f = 300e9;          % THz / sub-THz style simulation

c = 3e8;

K = 1;

L = 1;

lambda = c/f;

```

```

d = lambda/2;

N_iter = 10;

[w_far_BS] = exhaustive_codebook(Nt);
[w_hierarchy_tra] = hierarchy_binary_codebook(Nt);

setup_encoder;
Codebook = hierarchy_conv_codebook(Nt, conv_encoder_conf);

SNR_dB_list = -10:2:10;
SNR_list = 10.^(SNR_dB_list/10);
SNR_len = length(SNR_dB_list);

r_map_thz = zeros(SNR_len,1);

t0 = clock;

for idx = 1:SNR_len
    SNR = SNR_list(idx);
    fprintf('THz MAP | SNR_dB = %.2f [%d/%d] | elapsed %.2f s\n', ...
        SNR_dB_list(idx), idx, SNR_len, etime(clock,t0));

    for i_iter = 1:N_iter
        for k = 1:K
            theta_DoA(1,k) = unidrnd(Nt);
            [H(:,k), h_BS(:,k)] = generate_channel(Nt, theta_DoA(1,k), d, lambda);
        end

        for k = 1:K
            [~, id_BS_opt(1,k)] = training_exhaustive(w_far_BS, H(:,k), 100000, Nt);
        end
    end
end

```



```

    for k = 1:K
        [~, id_BS_map(1,k)] = training_hierarchy_map_thz(Codebook, H(:,k), SNR, Nt,
conv_encoder_conf);
        if id_BS_map(1,k) == id_BS_opt(1,k)
            r_map_thz(idx) = r_map_thz(idx) + 1;
        end
    end
end
end
end

r_map_thz = r_map_thz/(K*N_iter);

figure; plot(SNR_dB_list, r_map_thz, '-^', 'LineWidth', 1.6); grid on;
xlabel('SNR (dB)'); ylabel('Success Rate');
title('THz MAP Decoder at 300 GHz');

function [array_gain_map, id_BS_map] = training_hierarchy_map_thz(Codebook, H, SNR, Nt,
conv_encoder_conf)

    [array_gain_map, id_BS_map] = training_hierarchy_softconv(Codebook, H, SNR, Nt,
conv_encoder_conf);
end

```